

DOCKER

1.- Conceptos Básicos.

Docker es una aplicación que permite gestionar contenedores. Un contenedor no es más que la ejecución de un sistema operativo y un software específico para resolver un problema determinado. Es realmente una máquina virtual ejecutada en un entorno concreto. El “ordenador” es conocido como anfitrión o HOST.

Se llama **imagen** al bloque que conforma el sistema operativo con el software específico. Se llama **contenedor** a la ejecución de una imagen con unos parámetros concretos.

Por ejemplo, una imagen para php contendría el sistema operativo (Ubuntu), un servidor web (apache) configurado con PHP (todos los paquetes necesarios), phpmyadmin, etc. Un contenedor sería ejecutar esa imagen con el nombre mi-php, puerto 8080 y con una red virtual concreta.

Una imagen viene identificada por dos elementos nombre:tag. La tag indica la versión tal como la haya definido el proveedor. Normalmente, siempre hay una que es latest, que representa la última en ese momento. Por ejemplo, podrían ser imágenes nginx:latest, nginx, nginx:1.0, nginx/unit:1.29, etc Pueden consultarse imágenes existentes en docker hub (<https://hub.docker.com/>)

Toda imagen viene identificada por un código único (alfanumérico) y un nombre (establecido por nosotros o de forma automática).

Como he dicho, los contenedores se ejecutan dentro de un entorno específico y pueden conectarse entre ellos mediante las redes virtuales, redes independientes de las que tiene el HOST.

Además, se usan volúmenes como mecanismo de almacenamiento de datos persistentes y carpetas compartidas entre el HOST y el contenedor.

Se puede trabajar con docker de forma gráfica (en Windows con Docker Desktop) o en línea de comandos (en cualquier entorno con el comando **docker**). En este documento usaré el comando docker.

Escribiendo directamente el comando docker sin parámetros obtenemos ayuda sobre las opciones posibles.

2.- Ejecución de contenedores.

El elemento principal de docker es el contenedor. Para la creación de un contenedor usamos el comando **docker container run imagen** con diversos parámetros (se suele omitir container: **docker run imagen**). A continuación muestro varias ejecuciones usando como imagen nginx:

Ejecutar la imagen nginx
`docker container run nginx`

```
docker run nginx
```

Ejecutar la imagen nginx poniéndole como nombre al contenedor mi-nginx

```
docker run --name mi-nginx imagen
```

Ejecutar una imagen nginx mapeando el puerto 80 expuesto por nginx al puerto 8080 del host.

```
docker run -p 8080:80 nginx
```

Ejecutar una imagen nginx dejando libre la línea de comandos (en segundo plano)

```
docker run -d nginx
```

Los anteriores se pueden combinar en un solo comando:

```
docker run -d -p 8080 --name mi-nginx nginx
```

Lanzar una imagen Ubuntu ejecutando el interprete bash una vez arranque

```
docker run -it ubuntu bash
```

3.- Gestión de contenedores

Los contenedores pueden arrancarse, pararse, eliminarse, etc. Los contenedores tienen un ID único que puede ser usado, además de con el nombre, para gestionar el mismo.

Las ordenes son del tipo docker container aunque pueden ser simplificadas eliminando container.

Listar los contenedores funcionando actualmente

```
docker ps
```

```
docker container ps
```

Listar todos los contenedores existentes (funcionando o no)

```
docker ps -a
```

Parar el contenedor mi-nginx.

```
docker stop mi-nginx
```

Arrancar el contenedor mi-nginx

```
docker start mi-nginx
```

Ejecutar comando bash sobre el contenedor mi-nginx en un terminal interactivo.

```
docker exec -it mi-nginx bash
```

Ver toda la información del contenedor mi-nginx

```
docker inspect mi-nginx
```

4.- Almacenamiento

Docker permite trabajar en los contenedores con almacenamiento externo al propio contenedor. Nos permite trabajar de dos formas: con enlaces a carpetas del host (bind mount) y con volúmenes de docker

Bind mount:

Enlazan una carpeta del host a una carpeta del contenedor. Se utiliza la opción mount de docker run.

Permite definir el enlace usando dos sintaxis:

```
docker run --mount type=bind,src=<carpeta_host>,dst=<carpeta_contenedor>
docker run --volume <carpeta_host>:<carpeta_contenedor>
```

Se puede indicar src o source y dst o target.

Montar la carpeta local C:/web/sitiosWeb/html en la carpeta de la imagen nginx /usr/share/nginx/html que es donde se ubica la raíz del sitio.

```
docker run -d -p 8080:80 --name mi-nginx --mount
type=bind,src=C:/web/sitiosWeb/html,dst=/usr/share/nginx/html nginx:latest

docker run -d -p 8080:80 --name mi-nginx -v
C:/web/sitiosWeb/html:/usr/share/nginx/html nginx:latest
```

Volumenes

Un volumen es un espacio dentro del entorno de docker que puede ser compartido entre diferentes contenedores.

Enlazar el volumen volumen-nginx a la carpeta del contenedor usr/share/nginx/html

```
docker run -d -p 8080:80 --name mi-nginx --mount type=volume,src=volumen-
nginx,dst=/usr/share/nginx/html nginx:latest
docker run -d -p 8080:80 --name mi-nginx -v volumen-nginx:/usr/share/nginx/html
nginx:latest
```

Si queremos copiar en un volumen, lo que hacemos es copiar en la carpeta de un contenedor enlazada a ese volumen. Para copiar usamos el comando docker cp

```
docker cp C:\web\sitiosWeb\html\ mi-nginx:/usr/share/nginx/html
```

Los volúmenes pueden ser gestionados mediante la serie de comandos docker volume:

- docker volumen create volumen: crear volumen
- docker volume ls: listar los volúmenes
- docker volume inspect volumen: mostrar información de volumen
- docker volumen rm volumen: borrar un volumen

5.- Redes

Docker utiliza diferentes drivers de red para controlar cómo se comunican los contenedores entre sí y con el mundo exterior. Los tres más utilizados son:

- **Bridge (Puente):** Es la red por defecto. Crea una red privada virtual dentro de tu máquina que aísla los contenedores del exterior. Los contenedores pueden hablar entre

ellos mediante su IP o su nombre, pero para acceder a ellos desde fuera debes mapear puertos (ej. -p 8080:80).]

- **Host:** Elimina el aislamiento de red. El contenedor utiliza directamente la pila de red de la máquina anfitriona. Ofrece el mejor rendimiento, pero requiere gestionar los puertos cuidadosamente para evitar conflictos.
- **None (Ninguna):** Desconecta el contenedor por completo de cualquier red. Útil para máxima seguridad o para tareas aisladas de procesamiento por lotes donde la conectividad de red no es necesaria.

Normalmente se usará bridge que es la red por defecto. De esta forma, cuando usamos docker run el contenedor creado se enlaza directamente a bridge. Podemos ver la dirección ip asignada consultando la información con docker inspect.

Además, podemos definir nuestras propias redes privadas y enlazar los contenedores a esas redes.

Crear la red mi-red

```
docker network create mi-red
```

Conectar el contenedor mi-nginx a la red mi-red

```
docker run -d -p 8080:80 --network mi-red --name mi-nginx nginx:latest
```

Consultar información de la red

```
docker network inspect mi-red
```

Conectar la red mi-red al contenedor mi-nginx1

```
docker network connect mi-red mi-nginx1
```

Desconectar la red mi-redna red del contenedor mi-nginx

```
docker network disconnect mi-red mi-nginx
```

Borrar la red mi-red

```
docker network rm mi-red
```

6.- Gestión de imágenes

Todos los comandos relacionados con las imágenes los podemos encontrar bajo la opción docker image.

Listar imágenes existentes

```
docker image ls
```

Obtener la imagen de nginx:latest

```
docker image pull nginx:latest
```

```
docker pull nginx:latest
```

Borrar la imagen nginx:latest del repositorio

```
docker image rm nginx:latest
```

Crear la imagen mi-nginx-imag a partir del contenedor mi-nginx (con su información actual)

7. Docker Compose.

Docker compose permite la creación y gestión de contenedores de forma automática.

Se basa en la creación de un fichero en el que establecemos todos los contenedores, redes, volúmenes, etc que se van a necesitar con todas las dependencias y requisitos.

Por defecto, se usa el fichero docker-compose.yaml con una estructura específica para definir todos los elementos.

Los comandos relacionados con docker compose son de la forma **docker compose**.

Puede indicarse el fichero docker-compose mediante la opción `-f nombre_fichero`.

Los comandos principales son:

- `docker compose up`: carga el fichero docker-compose.yaml y lo ejecuta creando los servicios (contenedores, redes, volúmenes....)
- `docker compose up -d`: Carga y ejecuta el fichero en segundo plano, por lo que se puede seguir trabajando en la terminal.
- `docker compose stop|start|restart`: Para|inicia|reinicia la ejecución de los contenedores
- `docker compose down`: Elimina los servicios.